

Celebal Excellence Internship

Data Engineer

Name	Yadnyesh Sawant
Student ID	CT_CSI_DE_1015
Domain	Data Engineering
Batch	Batch 1
Start Date	15/6/2026
Status	Active
Contact No	8888047246
Email ID	<u>1272250066@mitwpu.edu.in</u>
College Name	MIT-WPU
Stream	MCA
Passing Out Year	2027

CELEBAL EXCELLENCE INTERNSHIP DATA ENGINEER

Week 2 Assignment

Task 1 from LMS Portal

Problem Statement

SQL-based data analysis using filtering, aggregation, and basic business queries

- **Objective:** Analyze sales data using SQL by applying filtering, aggregation, sorting, and business queries to generate meaningful insights

Step-by-Step Instructions

1. **Load the Dataset:** Import the Superstore dataset into a SQL database.
2. **Explore the Data:** View the table schema and display sample records.
3. **Apply WHERE Filters:** Filter data based on region, category, order date, and sales.
4. **Perform Aggregations:** Use GROUP BY to calculate total sales, total quantity, and average sales.
5. **Sort and Limit Results:** Identify top products, top categories, and highest sales using ORDER BY and LIMIT.
6. **Solve Business Use Cases:** Analyze monthly sales trends, identify top customers, and detect duplicate records.
7. **Validate Results:** Verify row counts, check for duplicates, and perform basic data quality checks.

Resources

- **Dataset:** <https://www.kaggle.com/datasets/vivek468/superstore-dataset-final>
- **Assignment Document:** https://celebaltech.sharepoint.com/:w:/s/Celebal-LMS/IQDQ_Co-E08_RZo3eKklxGR-AQ09RkM5df72wSBZjaL9LYc?e=04TLA8

Outputs

- SQL Script or Jupyter Notebook
- Query Results
- Brief Insights
- Word file with screenshots as suggested in the Saturday's 27-06-2026 meeting.

1. **Load the Dataset:** Import the Superstore dataset into a SQL database.
Used the word file as reference for the table and data.
2. **Explore the Data:** View the table schema and display sample records.

DESCRIBE customers;

Field	Type	Null	Key	Default	Extra
customer_id	int	NO	PRI	NULL	
first_name	varchar(50)	NO		NULL	
last_name	varchar(50)	NO		NULL	
email	varchar(100)	NO	UNI	NULL	
city	varchar(50)	NO	MUL	NULL	
state	varchar(50)	NO	MUL	NULL	
join_date	date	NO		NULL	
is_premium	tinyint(1)	YES		0	

DESCRIBE products;

Field	Type	Null	Key	Default	Extra
product_id	int	NO	PRI	NULL	
product_name	varchar(100)	NO		NULL	
category	varchar(50)	NO	MUL	NULL	
brand	varchar(50)	NO		NULL	
unit_price	decimal(10,2)	NO		NULL	
stock_qty	int	NO		0	

DESCRIBE orders;

Field	Type	Null	Key	Default	Extra
order_id	int	NO	PRI	NULL	
customer_id	int	NO	MUL	NULL	
order_date	date	NO	MUL	NULL	
status	varchar(20)	NO	MUL	Pending	
total_amount	decimal(12,2)	NO		NULL	

DESCRIBE order_items;

Field	Type	Null	Key	Default	Extra
item_id	int	NO	PRI	NULL	
order_id	int	NO	MUL	NULL	
product_id	int	NO	MUL	NULL	
quantity	int	NO		NULL	
unit_price	decimal(10,2)	NO		NULL	
discount_pct	decimal(5,2)	YES		0.00	

3. **Apply WHERE Filters:** Filter data based on region, category, order date, and sales.

SELECT * FROM customers WHERE state = 'Maharashtra';

customer_id	first_name	last_name	email	city	state	join_date	is_premium
101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1

SELECT * FROM products WHERE category = 'Electronics';

product_id	product_name	category	brand	unit_price	stock_qty
201	Wireless Earbuds	Electronics	BoAt	1499.00	250
203	Smart Watch	Electronics	Noise	2999.00	150
205	Bluetooth Speaker	Electronics	JBL	3499.00	200
207	Laptop Stand	Electronics	AmazonBasics	899.00	180

SELECT * FROM orders WHERE order_date BETWEEN '2024-08-01' AND '2024-08-31';

order_id	customer_id	order_date	status	total_amount
1001	101	2024-08-01	Delivered	4498.00
1002	102	2024-08-03	Delivered	799.00
1003	103	2024-08-05	Shipped	7498.00
1004	101	2024-08-10	Delivered	3499.00
1005	104	2024-08-12	Cancelled	2999.00
1006	105	2024-08-15	Delivered	5898.00
1007	106	2024-08-18	Pending	1299.00
1008	103	2024-08-20	Delivered	899.00
1009	107	2024-08-25	Shipped	6098.00
1010	108	2024-08-28	Delivered	1598.00
NULL	NULL	NULL	NULL	NULL

SELECT * FROM orders WHERE total_amount > 1000;

order_id	customer_id	order_date	status	total_amount
1001	101	2024-08-01	Delivered	4498.00
1003	103	2024-08-05	Shipped	7498.00
1004	101	2024-08-10	Delivered	3499.00
1005	104	2024-08-12	Cancelled	2999.00
1006	105	2024-08-15	Delivered	5898.00
1007	106	2024-08-18	Pending	1299.00
1009	107	2024-08-25	Shipped	6098.00
1010	108	2024-08-28	Delivered	1598.00
1011	102	2026-06-28	Pending	1598.00
1045	102	2026-06-28	Pending	1598.00
1054	102	2026-06-28	Pending	1598.00
NULL	NULL	NULL	NULL	NULL

SELECT * FROM customers WHERE is_premium = TRUE;

customer_id	first_name	last_name	email	city	state	join_date	is_premium
101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1
105	Vikram	Singh	vikram.s@email.com	Jaipur	Rajasthan	2024-05-12	1
107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

4. **Perform Aggregations:** Use GROUP BY to calculate total sales, total quantity, and average sales.

SELECT status,SUM(total_amount) AS total_sales FROM orders GROUP BY status;

status	total_sales
Cancelled	2999.00
Delivered	17191.00
Pending	6093.00
Shipped	13596.00

SELECT product_id,SUM(quantity) AS total_quantity FROM order_item GROUP BY product_id;

product_id	total_quantity
201	3
202	1
203	2
204	2
205	2
206	5
207	2
208	6

SELECT AVG(total_amount) AS average_order_value FROM orders;

average_order_value
3067.615385

```
SELECT customer_id,SUM(total_amount) AS total_sales FROM orders GROUP BY customer_id;
```

customer_id	total_sales
101	7997.00
102	5593.00
103	8397.00
104	2999.00
105	5898.00
106	1299.00
107	6098.00
108	1598.00

```
SELECT category,COUNT(*) AS total_products FROM products GROUP BY category;
```

category	total_products
Clothing	2
Electronics	4
Home	2

5. **Sort and Limit Results:** Identify top products, top categories, and highest sales using ORDER BY and LIMIT.

```
SELECT * FROM products ORDER BY unit_price DESC LIMIT 5;
```

product_id	product_name	category	brand	unit_price	stock_qty
204	Running Shoes	Clothing	Nike	4599.00	120
205	Bluetooth Speaker	Electronics	JBL	3499.00	200
203	Smart Watch	Electronics	Noise	2999.00	150
201	Wireless Earbuds	Electronics	BoAt	1499.00	250
206	Bedsheet Set	Home	Spaces	1299.00	297
NULL	NULL	NULL	NULL	NULL	NULL

```
SELECT * FROM orders ORDER BY total_amount DESC LIMIT 5;
```

order_id	customer_id	order_date	status	total_amount
1003	103	2024-08-05	Shipped	7498.00
1009	107	2024-08-25	Shipped	6098.00
1006	105	2024-08-15	Delivered	5898.00
1001	101	2024-08-01	Delivered	4498.00
1004	101	2024-08-10	Delivered	3499.00
NULL	NULL	NULL	NULL	NULL

```
SELECT customer_id,SUM(total_amount) AS total_spent FROM orders GROUP BY customer_id ORDER BY total_spent DESC LIMIT 5;
```

customer_id	total_spent
103	8397.00
101	7997.00
107	6098.00
105	5898.00
102	5593.00

```
SELECT category,COUNT(*) AS total_products FROM products GROUP BY category ORDER BY total_products DESC;
```

category	total_products
Electronics	4
Clothing	2
Home	2

6. **Solve Business Use Cases:** Analyze monthly sales trends, identify top customers, and detect duplicate records.

```
SELECT YEAR(order_date) AS Year, MONTH(order_date) AS Month, SUM(total_amount)
AS Monthly_Sales FROM orders GROUP BY YEAR(order_date), MONTH(order_date) ORDER
BY Year, Month;
```

Year	Month	Monthly_Sales
2024	8	35085.00
2026	6	4794.00

```
SELECT c.customer_id, CONCAT(c.first_name, ' ',c.last_name) AS Customer_Name,
SUM(o.total_amount) AS Total_Spent FROM customers c JOIN orders o ON
c.customer_id=o.customer_id GROUP BY c.customer_id, Customer_Name ORDER BY
Total_Spent DESC LIMIT 5;
```

customer_id	Customer_Name	Total_Spent
103	Rohan Gupta	8397.00
101	Aarav Sharma	7997.00
107	Karan Mehta	6098.00
105	Vikram Singh	5898.00
102	Priya Patel	5593.00

```
SELECT p.product_id, p.product_name, SUM(oi.quantity) AS Total_Quantity_Sold
FROM products p JOIN order_items oi ON p.product_id=oi.product_id GROUP BY
p.product_id, p.product_name ORDER BY Total_Quantity_Sold DESC;
```

product_id	product_name	Total_Quantity_Sold
208	Cushion Covers (Set)	6
206	Bedsheet Set	5
201	Wireless Earbuds	3
203	Smart Watch	2
204	Running Shoes	2
205	Bluetooth Speaker	2
207	Laptop Stand	2
202	Cotton T-Shirt	1

```
SELECT email, COUNT(*) AS Duplicate_Count FROM customers GROUP BY email HAVING
COUNT(*)>1;
```

email	Duplicate_Count
-------	-----------------

```
SELECT order_id, COUNT(product_id) AS Number_Of_Products FROM order_items GROUP
BY order_id HAVING COUNT(product_id)>1;
```

order_id	Number_Of_Products
1001	2
1003	2
1006	2
1009	2
1010	2
1011	6

7. **Validate Results:** Verify row counts, check for duplicates, and perform basic data quality checks.

```
SELECT COUNT(*) AS Total_Customers FROM customers;
```

Total_Customers
8

```
SELECT COUNT(*) AS Total_Products FROM products;
```

Total_Products
8

```
SELECT COUNT(*) AS Total_Orders FROM orders;
```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
Total_Orders			
13			

```
SELECT COUNT(*) AS Total_Order_Items FROM order_items;
```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
Total_Order_Items			
21			

```
SELECT * FROM customers WHERE email IS NULL;
```

Result Grid

Filter Rows:

Edit:

Export/Import:

Wrap Cell Content:

	customer_id	first_name	last_name	email	city	state	join_date	is_premium
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

```
SELECT * FROM products WHERE stock_qty < 0;
```

Result Grid

Filter Rows:

Edit:

Export/Import:

Wrap Cell Content:

	product_id	product_name	category	brand	unit_price	stock_qty
*	NULL	NULL	NULL	NULL	NULL	NULL

```
SELECT * FROM orders WHERE total_amount < 0;
```

Result Grid	Filter Rows:	Edit:	Export/Import:	Wrap Cell Content:
order_id	customer_id	order_date	status	total_amount
*	NULL	NULL	NULL	NULL

```
SELECT oi.* FROM order_items oi LEFT JOIN orders o ON oi.order_id=o.order_id
WHERE o.order_id IS NULL;
```

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

item_id	order_id	product_id	quantity	unit_price	discount_pct
---------	----------	------------	----------	------------	--------------

```
SELECT oi.* FROM order_items oi LEFT JOIN products p ON
oi.product_id=p.product_id WHERE p.product_id IS NULL;
```

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

item_id	order_id	product_id	quantity	unit_price	discount_pct
---------	----------	------------	----------	------------	--------------

```
SELECT product_id, COUNT(*) AS Duplicate_Count FROM products GROUP BY product_id
HAVING COUNT(*)>1;
```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
product_id	Duplicate_Count		

Task 2 from Word File

```
use week2_assignment;
CREATE TABLE customers (
    customer_id    INT            PRIMARY KEY,
    first_name     VARCHAR(50)    NOT NULL,
    last_name      VARCHAR(50)    NOT NULL,
    email          VARCHAR(100)   UNIQUE NOT NULL,
    city           VARCHAR(50)    NOT NULL,
    state          VARCHAR(50)    NOT NULL,
    join_date      DATE           NOT NULL,
    is_premium     BOOLEAN        DEFAULT FALSE
);

-- Index for filtering by city/state
CREATE INDEX idx_customers_city ON customers(city);
CREATE INDEX idx_customers_state ON customers(state);

CREATE TABLE products (
    product_id     INT            PRIMARY KEY,
    product_name   VARCHAR(100)   NOT NULL,
    category       VARCHAR(50)    NOT NULL,
    brand          VARCHAR(50)    NOT NULL,
    unit_price     DECIMAL(10,2)  NOT NULL CHECK (unit_price > 0),
    stock_qty      INT            NOT NULL DEFAULT 0 CHECK (stock_qty >= 0)
);

-- Index for filtering by category
CREATE INDEX idx_products_category ON products(category);

CREATE TABLE orders (
    order_id       INT            PRIMARY KEY,
    customer_id    INT            NOT NULL,
    order_date     DATE           NOT NULL,
    status         VARCHAR(20)    NOT NULL DEFAULT 'Pending'
                        CHECK (status IN ('Pending', 'Shipped', 'Delivered', 'Cancelled')),
    total_amount   DECIMAL(12,2)  NOT NULL CHECK (total_amount >= 0),

    FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
);

-- Index for date-based filtering and sorting
CREATE INDEX idx_orders_date ON orders(order_date);
CREATE INDEX idx_orders_status ON orders(status);

CREATE TABLE order_items (
    item_id        INT            PRIMARY KEY,
    order_id       INT            NOT NULL,
    product_id     INT            NOT NULL,
    quantity       INT            NOT NULL CHECK (quantity > 0),
    unit_price     DECIMAL(10,2)  NOT NULL CHECK (unit_price > 0),
    discount_pct   DECIMAL(5,2)   DEFAULT 0 CHECK (discount_pct BETWEEN 0 AND 100),

    FOREIGN KEY (order_id) REFERENCES orders(order_id),
    FOREIGN KEY (product_id) REFERENCES products(product_id)
);
```

Output				
Action Output				
#	Time	Action	Message	Duration / Fetch
1	12:39:44	use week2_assignment	0 row(s) affected	0.031 sec
2	12:39:54	use week2_assignment	0 row(s) affected	0.000 sec
3	12:39:54	CREATE TABLE customers (customer_id INT PRIMARY KEY, first_name VARCHAR(50) NOT NULL, last_name ...	0 row(s) affected	0.234 sec
4	12:39:54	CREATE INDEX idx_customers_city ON customers(city)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.094 sec
5	12:39:54	CREATE INDEX idx_customers_state ON customers(state)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.047 sec
6	12:39:54	CREATE TABLE products (product_id INT PRIMARY KEY, product_name VARCHAR(100) NOT NULL, category ...	0 row(s) affected	0.079 sec
7	12:39:54	CREATE INDEX idx_products_category ON products(category)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.062 sec
8	12:39:54	CREATE TABLE orders (order_id INT PRIMARY KEY, customer_id INT NOT NULL, order_date DATE ...	0 row(s) affected	0.141 sec
9	12:39:54	CREATE INDEX idx_orders_date ON orders(order_date)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.172 sec
10	12:39:55	CREATE INDEX idx_orders_status ON orders(status)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.125 sec
11	12:39:55	CREATE TABLE order_items (item_id INT PRIMARY KEY, order_id INT NOT NULL, product_id INT ...	0 row(s) affected	0.172 sec


```
-- ===== INSERT: customers =====
```

```
INSERT INTO customers VALUES
```

```
(101, 'Aarav', 'Sharma', 'aarav.s@email.com', 'Mumbai', 'Maharashtra', '2024-01-15', TRUE),
(102, 'Priya', 'Patel', 'priya.p@email.com', 'Ahmedabad', 'Gujarat', '2024-02-20',
FALSE),
(103, 'Rohan', 'Gupta', 'rohan.g@email.com', 'Delhi', 'Delhi', '2024-03-10', TRUE),
(104, 'Sneha', 'Reddy', 'sneha.r@email.com', 'Hyderabad', 'Telangana', '2024-04-05',
FALSE),
(105, 'Vikram', 'Singh', 'vikram.s@email.com', 'Jaipur', 'Rajasthan', '2024-05-12', TRUE),
(106, 'Ananya', 'Iyer', 'ananya.i@email.com', 'Chennai', 'Tamil Nadu', '2024-06-18',
FALSE),
(107, 'Karan', 'Mehta', 'karan.m@email.com', 'Pune', 'Maharashtra', '2024-07-22', TRUE),
(108, 'Divya', 'Nair', 'divya.n@email.com', 'Kochi', 'Kerala', '2024-08-30',
FALSE);
```

```
-- ===== INSERT: products =====
```

```
INSERT INTO products VALUES
```

```
(201, 'Wireless Earbuds', 'Electronics', 'BoAt', 1499.00, 250),
(202, 'Cotton T-Shirt', 'Clothing', 'Levi's', 799.00, 500),
(203, 'Smart Watch', 'Electronics', 'Noise', 2999.00, 150),
(204, 'Running Shoes', 'Clothing', 'Nike', 4599.00, 120),
(205, 'Bluetooth Speaker', 'Electronics', 'JBL', 3499.00, 200),
(206, 'Bedsheet Set', 'Home', 'Spaces', 1299.00, 300),
(207, 'Laptop Stand', 'Electronics', 'AmazonBasics', 899.00, 180),
(208, 'Cushion Covers (Set)', 'Home', 'HomeCenter', 599.00, 400);
```

```
-- ===== INSERT: orders =====
```

```
INSERT INTO orders VALUES
```

```
(1001, 101, '2024-08-01', 'Delivered', 4498.00),
(1002, 102, '2024-08-03', 'Delivered', 799.00),
(1003, 103, '2024-08-05', 'Shipped', 7498.00),
(1004, 101, '2024-08-10', 'Delivered', 3499.00),
(1005, 104, '2024-08-12', 'Cancelled', 2999.00),
(1006, 105, '2024-08-15', 'Delivered', 5898.00),
(1007, 106, '2024-08-18', 'Pending', 1299.00),
(1008, 103, '2024-08-20', 'Delivered', 899.00),
(1009, 107, '2024-08-25', 'Shipped', 6098.00),
(1010, 108, '2024-08-28', 'Delivered', 1598.00);
```

```
-- ===== INSERT: order_items =====
```

```
INSERT INTO order_items VALUES
```

```
(5001, 1001, 201, 2, 1499.00, 0),
(5002, 1001, 207, 1, 899.00, 10),
(5003, 1002, 202, 1, 799.00, 0),
(5004, 1003, 203, 1, 2999.00, 0),
(5005, 1003, 204, 1, 4599.00, 5),
(5006, 1004, 205, 1, 3499.00, 0),
(5007, 1005, 203, 1, 2999.00, 0),
(5008, 1006, 201, 1, 1499.00, 10),
(5009, 1006, 204, 1, 4599.00, 5),
(5010, 1007, 206, 1, 1299.00, 0),
(5011, 1008, 207, 1, 899.00, 0),
(5012, 1009, 205, 1, 3499.00, 0),
(5013, 1009, 208, 2, 599.00, 15),
(5014, 1010, 206, 1, 1299.00, 0),
(5015, 1010, 208, 1, 599.00, 0);
```

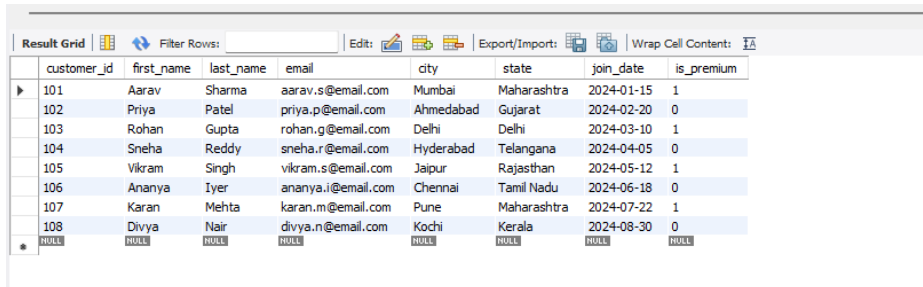
#	Time	Action	Message	Duration / Fetch
1	12:48:57	INSERT INTO customers VALUES (101, 'Aarav', 'Sharma', 'aarav.s@email.com', 'Mumbai', 'Maharashtra', '2024-01-15', TRUE), (102, 'Priya', 'Patel', 'priya.p@email.com', 'Ahmedabad', 'Gujarat', '2024-02-20', FALSE), (103, 'Rohan', 'Gupta', 'rohan.g@email.com', 'Delhi', 'Delhi', '2024-03-10', TRUE), (104, 'Sneha', 'Reddy', 'sneha.r@email.com', 'Hyderabad', 'Telangana', '2024-04-05', FALSE), (105, 'Vikram', 'Singh', 'vikram.s@email.com', 'Jaipur', 'Rajasthan', '2024-05-12', TRUE), (106, 'Ananya', 'Iyer', 'ananya.i@email.com', 'Chennai', 'Tamil Nadu', '2024-06-18', FALSE), (107, 'Karan', 'Mehta', 'karan.m@email.com', 'Pune', 'Maharashtra', '2024-07-22', TRUE), (108, 'Divya', 'Nair', 'divya.n@email.com', 'Kochi', 'Kerala', '2024-08-30', FALSE);	8 row(s) affected Records: 8 Duplicates: 0 Warnings: 0	0.015 sec
2	12:48:57	INSERT INTO products VALUES (201, 'Wireless Earbuds', 'Electronics', 'BoAt', 1499.00, 250), (202, 'Cotton T-Shirt', 'Clothing', 'Levi's', 799.00, 500), (203, 'Smart Watch', 'Electronics', 'Noise', 2999.00, 150), (204, 'Running Shoes', 'Clothing', 'Nike', 4599.00, 120), (205, 'Bluetooth Speaker', 'Electronics', 'JBL', 3499.00, 200), (206, 'Bedsheet Set', 'Home', 'Spaces', 1299.00, 300), (207, 'Laptop Stand', 'Electronics', 'AmazonBasics', 899.00, 180), (208, 'Cushion Covers (Set)', 'Home', 'HomeCenter', 599.00, 400);	8 row(s) affected Records: 8 Duplicates: 0 Warnings: 0	0.000 sec
3	12:48:57	INSERT INTO orders VALUES (1001, 101, '2024-08-01', 'Delivered', 4498.00), (1002, 102, '2024-08-03', 'Delivered', 799.00), (1003, 103, '2024-08-05', 'Shipped', 7498.00), (1004, 101, '2024-08-10', 'Delivered', 3499.00), (1005, 104, '2024-08-12', 'Cancelled', 2999.00), (1006, 105, '2024-08-15', 'Delivered', 5898.00), (1007, 106, '2024-08-18', 'Pending', 1299.00), (1008, 103, '2024-08-20', 'Delivered', 899.00), (1009, 107, '2024-08-25', 'Shipped', 6098.00), (1010, 108, '2024-08-28', 'Delivered', 1598.00);	10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0	0.016 sec
4	12:48:57	INSERT INTO order_items VALUES (5001, 1001, 201, 2, 1499.00, 0), (5002, 1001, 207, 1, 899.00, 10), (5003, 1002, 202, 1, 799.00, 0), (5004, 1003, 203, 1, 2999.00, 0), (5005, 1003, 204, 1, 4599.00, 5), (5006, 1004, 205, 1, 3499.00, 0), (5007, 1005, 203, 1, 2999.00, 0), (5008, 1006, 201, 1, 1499.00, 10), (5009, 1006, 204, 1, 4599.00, 5), (5010, 1007, 206, 1, 1299.00, 0), (5011, 1008, 207, 1, 899.00, 0), (5012, 1009, 205, 1, 3499.00, 0), (5013, 1009, 208, 2, 599.00, 15), (5014, 1010, 206, 1, 1299.00, 0), (5015, 1010, 208, 1, 599.00, 0);	15 row(s) affected Records: 15 Duplicates: 0 Warnings: 0	0.015 sec

Section A – SQL Basics (SELECT, Constraints, Primary Keys)

These questions test your understanding of basic data retrieval, table structure, and database constraints.

Q1. Write a query to display all columns and rows from the customer's table.

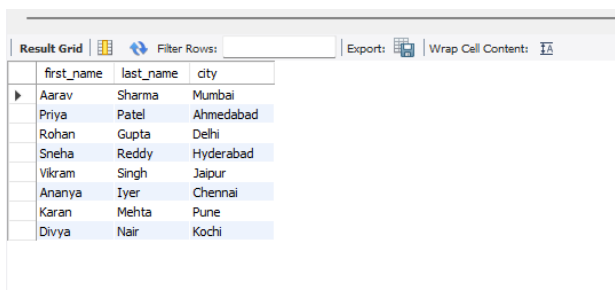
```
SELECT * FROM customers;
```



	customer_id	first_name	last_name	email	city	state	join_date	is_premium
▶	101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
	102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat	2024-02-20	0
	103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1
	104	Sneha	Reddy	sneha.r@email.com	Hyderabad	Telangana	2024-04-05	0
	105	Vikram	Singh	vikram.s@email.com	Jaipur	Rajasthan	2024-05-12	1
	106	Ananya	Iyer	ananya.i@email.com	Chennai	Tamil Nadu	2024-06-18	0
	107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1
	108	Divya	Nair	divya.n@email.com	Kochi	Kerala	2024-08-30	0
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Q2. Retrieve only the first_name, last_name, and city of all customers.

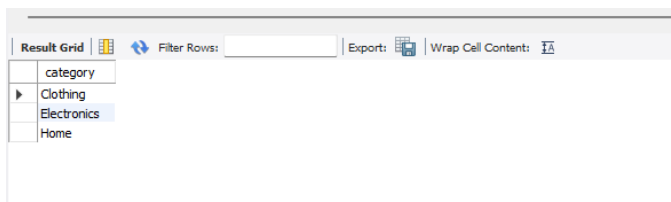
```
SELECT first_name,last_name,city FROM customers;
```



	first_name	last_name	city
▶	Aarav	Sharma	Mumbai
	Priya	Patel	Ahmedabad
	Rohan	Gupta	Delhi
	Sneha	Reddy	Hyderabad
	Vikram	Singh	Jaipur
	Ananya	Iyer	Chennai
	Karan	Mehta	Pune
	Divya	Nair	Kochi

Q3. List all unique categories available in the products table.

```
SELECT DISTINCT category FROM products;
```



	category
▶	Clothing
	Electronics
	Home

Q4. Identify the Primary Key of each table in the schema. Explain why a Primary Key must be unique and NOT NULL.

Answer:

The **Primary Key** of each table is defined in its CREATE TABLE statement. The primary keys in the schema are:

- **Customer** table: customer_id
- **Product** table: product_id
- **Orders** table: order_id
- **Order_Items** table: item_id

A **Primary Key** must be **unique** and **NOT NULL** because it uniquely identifies each record in a table. The Uniqueness prevents duplicate records, while the NOT NULL constraint ensures that every row has a valid identifier and the primary key is not left empty while inserting the data to the table. This helps maintain data integrity and enables accurate relationships between tables through foreign keys.

Q5. What constraints are applied to the email column in the customers table? What would happen if you tried to insert a duplicate email?

Answer:

The email column in the customers table has the following constraints:

- **UNIQUE** – Ensures that each email address is unique and cannot be repeated in the table.
- **NOT NULL** – Ensures that every customer must have an email address, NULL values are not allowed.

If you try to insert a duplicate email address, the database will reject the operation and return a **UNIQUE constraint violation** error. This prevents duplicate customer records and helps maintain data integrity.

Example if duplicate email inserted:

Output				
Action Output				
#	Time	Action	Message	
1	13:03:46	INSERT INTO customers VALUES (109, 'Rahul', 'Verma', 'aarav.s@email.com...	Error Code: 1062. Duplicate entry 'aarav.s@email.com' for key 'customers.email'	

Q6. Try inserting a product with unit_price = -50. What happens and which constraint prevents it? Write both the INSERT statement and explain the error.

```
INSERT INTO products VALUES (209, 'USB Cable', 'Electronics', 'Boat', -50, 100);
```

Output				
Action Output				
#	Time	Action	Message	
1	13:05:18	INSERT INTO products VALUES (209, 'USB Cable', 'Electronics', 'Boat', -50, 100)	Error Code: 3819. Check constraint 'products_chk_1' is violated.	

Explanation:

When this statement is executed, the database rejects the insertion and displays the following error:

Error Code: 3819. Check constraint 'products_chk_1' is violated.

This occurs because the unit_price column has the following constraint:

CHECK (unit_price > 0)

The value -50 does not satisfy this condition since the unit price must be greater than zero. The **CHECK constraint** (products_chk_1) prevents invalid data from being inserted into the table, ensuring that all product prices are positive or greater than 0.

Section B – Filtering & Optimization (WHERE, Indexes)

These questions test your ability to filter data effectively and understand how indexes improve query performance.

Q7. Retrieve all orders with status = 'Delivered'.

```
SELECT * FROM orders WHERE status = 'Delivered';
```

order_id	customer_id	order_date	status	total_amount
1001	101	2024-08-01	Delivered	4498.00
1002	102	2024-08-03	Delivered	799.00
1004	101	2024-08-10	Delivered	3499.00
1006	105	2024-08-15	Delivered	5898.00
1008	103	2024-08-20	Delivered	899.00
1010	108	2024-08-28	Delivered	1598.00
NULL	NULL	NULL	NULL	NULL

Q8. Find all products in the 'Electronics' category with a unit_price greater than ₹2000.

```
SELECT * FROM products WHERE category = 'Electronics' AND unit_price > 2000;
```

product_id	product_name	category	brand	unit_price	stock_qty
203	Smart Watch	Electronics	Noise	2999.00	150
205	Bluetooth Speaker	Electronics	JBL	3499.00	200
NULL	NULL	NULL	NULL	NULL	NULL

Q9. List all customers who joined in the year 2024 and belong to the state 'Maharashtra'.

```
SELECT * FROM customers WHERE join_date BETWEEN '2024-01-01' AND '2024-12-31' AND state = 'Maharashtra';
```

customer_id	first_name	last_name	email	city	state	join_date	is_premium
101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Q10. Find all orders placed between '2024-08-10' and '2024-08-25' (inclusive) that are NOT cancelled.

```
SELECT * FROM orders WHERE order_date BETWEEN '2024-08-10' AND '2024-08-25' AND status <> 'Cancelled';
```

order_id	customer_id	order_date	status	total_amount
1004	101	2024-08-10	Delivered	3499.00
1006	105	2024-08-15	Delivered	5898.00
1007	106	2024-08-18	Pending	1299.00
1008	103	2024-08-20	Delivered	899.00
1009	107	2024-08-25	Shipped	6098.00
NULL	NULL	NULL	NULL	NULL

Q11. Explain what the index `idx_orders_date` does. How would it improve the performance of a query that filters orders by `order_date`? Write a sample query that would benefit from this index.

Answer:

The `idx_orders_date` index is created on the `order_date` column in the `orders` table.

```
CREATE INDEX idx_orders_date ON orders(order_date);
```

This index makes searching for orders by date faster. Instead of checking every row in the table, the database uses the index to directly find the matching records. This improves the performance of queries that filter orders based on the `order_date`, especially when the table has a large amount of data.

Example Query:

```
SELECT * FROM orders WHERE order_date BETWEEN '2024-08-10' AND '2024-08-25';
```

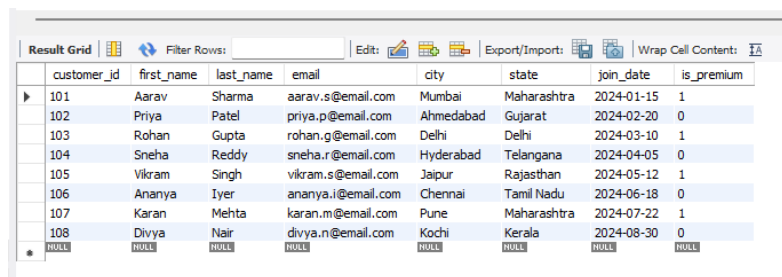
This query uses the `order_date` column in the `WHERE` clause, so the `idx_orders_date` index helps retrieve the matching records more quickly.

Q12. If you run: `SELECT * FROM customers WHERE YEAR(join_date) = 2024;` – would the index on `join_date` be used? Explain why or why not, and rewrite the query to be index-friendly (SARGable).

Answer:

No, the index on `join_date` would not be used because the `YEAR()` function is applied to the `join_date` column. When a function is used on an indexed column, the database cannot use the index properly and has to check every row in the table.

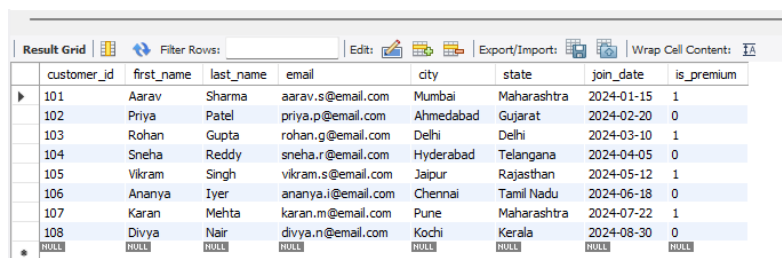
```
SELECT * FROM customers WHERE YEAR(join_date) = 2024;
```



customer_id	first_name	last_name	email	city	state	join_date	is_premium
101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat	2024-02-20	0
103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1
104	Sneha	Reddy	sneha.r@email.com	Hyderabad	Telangana	2024-04-05	0
105	Vikram	Singh	vikram.s@email.com	Jaipur	Rajasthan	2024-05-12	1
106	Ananya	Iyer	ananya.i@email.com	Chennai	Tamil Nadu	2024-06-18	0
107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1
108	Divya	Nair	divya.n@email.com	Kochi	Kerala	2024-08-30	0
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

A better way to write the query is:

```
SELECT * FROM customers WHERE join_date BETWEEN '2024-01-01' AND '2024-12-31';
```



customer_id	first_name	last_name	email	city	state	join_date	is_premium
101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat	2024-02-20	0
103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1
104	Sneha	Reddy	sneha.r@email.com	Hyderabad	Telangana	2024-04-05	0
105	Vikram	Singh	vikram.s@email.com	Jaipur	Rajasthan	2024-05-12	1
106	Ananya	Iyer	ananya.i@email.com	Chennai	Tamil Nadu	2024-06-18	0
107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1
108	Divya	Nair	divya.n@email.com	Kochi	Kerala	2024-08-30	0
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

This query is **SARGable** because it compares the `join_date` column directly without using any function. If there is an index on `join_date`, the database can use it to find the required records faster.

Section C – Aggregation (GROUP BY, SUM, COUNT, AVG, MIN, MAX)
 These questions test your ability to summarize and aggregate data.

Q13. Count the total number of orders in the orders table.

```
SELECT COUNT(*) AS total_orders FROM orders;
```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
	total_orders			
▶	10			

Q14. Find the total revenue (SUM of total_amount) from all 'Delivered' orders.

```
SELECT SUM(total_amount) AS total_revenue FROM orders WHERE status = 'Delivered';
```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
	total_revenue			
▶	17191.00			

Q15. Calculate the average unit_price of products in each category.

```
SELECT category, AVG(unit_price) AS average_price FROM products GROUP BY category;
```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
	category	average_price		
▶	Clothing	2699.000000		
	Electronics	2224.000000		
	Home	949.000000		

v

Q16. For each order status, find the count of orders and the total revenue. Sort the result by total revenue in descending order.

```
SELECT status, COUNT(*) AS order_count, SUM(total_amount) AS total_revenue FROM orders GROUP BY status ORDER BY total_revenue DESC;
```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
	status	order_count	total_revenue	
▶	Delivered	6	17191.00	
	Shipped	2	13596.00	
	Cancelled	1	2999.00	
	Pending	1	1299.00	

Q17. Find the most expensive (MAX) and cheapest (MIN) product in each category.

```
SELECT category, MAX(unit_price) AS highest_price, MIN(unit_price) AS lowest_price FROM products GROUP BY category;
```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
	category	highest_price	lowest_price	
▶	Clothing	4599.00	799.00	
	Electronics	3499.00	899.00	
	Home	1299.00	599.00	

Q18. List all product categories where the average unit_price is greater than ₹2000. (Hint: Use HAVING clause)

```
SELECT category, AVG(unit_price) AS average_price FROM products GROUP BY category HAVING  
AVG(unit_price) > 2000;
```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
category	average_price			
Clothing	2699.000000			
Electronics	2224.000000			

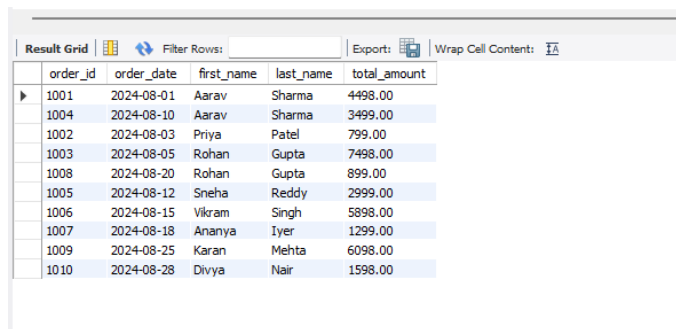
Section D – Joins & Relationships

These questions test your ability to combine data from multiple tables using different types of JOINS.

Q19. Write an INNER JOIN query to display each order along with the customer's first_name and last_name.

Show: order_id, order_date, first_name, last_name, total_amount.

```
SELECT o.order_id, o.order_date, c.first_name, c.last_name, o.total_amount FROM orders o
INNER JOIN customers c ON o.customer_id = c.customer_id;
```

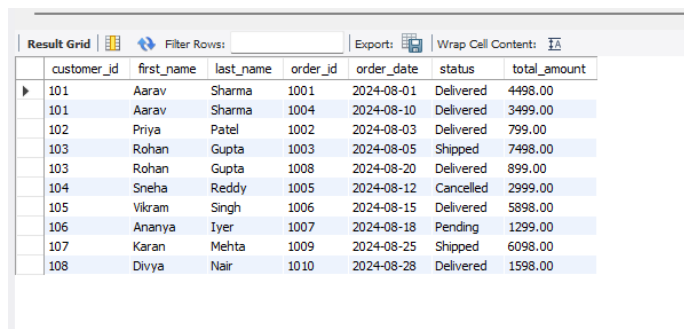


The screenshot shows a database query result grid with the following data:

order_id	order_date	first_name	last_name	total_amount
1001	2024-08-01	Aarav	Sharma	4498.00
1004	2024-08-10	Aarav	Sharma	3499.00
1002	2024-08-03	Priya	Patel	799.00
1003	2024-08-05	Rohan	Gupta	7498.00
1008	2024-08-20	Rohan	Gupta	899.00
1005	2024-08-12	Sneha	Reddy	2999.00
1006	2024-08-15	Vikram	Singh	5898.00
1007	2024-08-18	Ananya	Iyer	1299.00
1009	2024-08-25	Karan	Mehta	6098.00
1010	2024-08-28	Divya	Nair	1598.00

Q20. Using a LEFT JOIN, list ALL customers and their orders (if any). Customers with no orders should still appear with NULL values for order columns.

```
SELECT c.customer_id, c.first_name, c.last_name, o.order_id, o.order_date, o.status,
o.total_amount FROM customers c LEFT JOIN orders o ON c.customer_id = o.customer_id;
```



The screenshot shows a database query result grid with the following data:

customer_id	first_name	last_name	order_id	order_date	status	total_amount
101	Aarav	Sharma	1001	2024-08-01	Delivered	4498.00
101	Aarav	Sharma	1004	2024-08-10	Delivered	3499.00
102	Priya	Patel	1002	2024-08-03	Delivered	799.00
103	Rohan	Gupta	1003	2024-08-05	Shipped	7498.00
103	Rohan	Gupta	1008	2024-08-20	Delivered	899.00
104	Sneha	Reddy	1005	2024-08-12	Cancelled	2999.00
105	Vikram	Singh	1006	2024-08-15	Delivered	5898.00
106	Ananya	Iyer	1007	2024-08-18	Pending	1299.00
107	Karan	Mehta	1009	2024-08-25	Shipped	6098.00
108	Divya	Nair	1010	2024-08-28	Delivered	1598.00

Q21. Write a query using JOINS across three tables (orders → order_items → products) to show: order_id, product_name, quantity, unit_price, and discount_pct for each order item.

```
SELECT o.order_id, p.product_name, oi.quantity, oi.unit_price, oi.discount_pct
FROM orders o JOIN order_items oi ON o.order_id = oi.order_id JOIN products p
ON oi.product_id = p.product_id;
```

order_id	product_name	quantity	unit_price	discount_pct
1001	Wireless Earbuds	2	1499.00	0.00
1006	Wireless Earbuds	1	1499.00	10.00
1002	Cotton T-Shirt	1	799.00	0.00
1003	Smart Watch	1	2999.00	0.00
1005	Smart Watch	1	2999.00	0.00
1003	Running Shoes	1	4599.00	5.00
1006	Running Shoes	1	4599.00	5.00
1004	Bluetooth Speaker	1	3499.00	0.00
1009	Bluetooth Speaker	1	3499.00	0.00
1007	Bedsheet Set	1	1299.00	0.00
1010	Bedsheet Set	1	1299.00	0.00
1001	Laptop Stand	1	899.00	10.00
1008	Laptop Stand	1	899.00	0.00
1009	Cushion Covers (...)	2	599.00	15.00
1010	Cushion Covers (...)	1	599.00	0.00

Q22. Explain the difference between LEFT JOIN and RIGHT JOIN with an example from this schema. When would you use a FULL OUTER JOIN?

Answer:

- A **LEFT JOIN** returns all records from the left table and the matching records from the right table. If there is no matching record in the right table, the result will contain NULL values.

Example:

```
SELECT c.customer_id, c.first_name, o.order_id FROM customers c LEFT JOIN
orders o ON c.customer_id = o.customer_id;
This query shows all customers, even if they have not placed any orders.
```

- A **RIGHT JOIN** returns all records from the right table and the matching records from the left table. If there is no matching record in the left table, the result will contain NULL values.

Example:

```
SELECT c.customer_id, c.first_name, o.order_id FROM customers c
RIGHT JOIN orders o ON c.customer_id = o.customer_id;
This query shows all orders, even if there is no matching customer.
```

- A **FULL OUTER JOIN** returns all records from both tables. If a row does not have a matching row in the other table, the missing values are shown as NULL.
- A **FULL OUTER JOIN** is used when we want to see **all customers and all orders**, including customers who have not placed any orders and orders that do not have a matching customer.

Q23. Identify all Foreign Key relationships in the schema. Explain what would happen if you tried to insert an order with customer_id = 999 (which doesn't exist in customers).

Answer:

The schema has the following **Foreign Key** relationships:

1. orders.customer_id references customers.customer_id
2. order_items.order_id references orders.order_id
3. order_items.product_id references products.product_id

These foreign keys ensure that the related record exists in the parent table before a record can be inserted into the child table.

If you try to insert an order with customer_id = 999 and that customer does not exist in the customers table, the database will reject the insertion and display a **Foreign Key constraint** error.

Example:

```
INSERT INTO orders VALUES (101, 999, '2024-08-20', 'Pending', 500.00);
Since customer_id = 999 is not present in the customers table, the insert operation will fail. This helps maintain referential integrity by preventing orders from being linked to non-existent customers.
```

Section E – Advanced Concepts (CASE, ACID, Transactions)

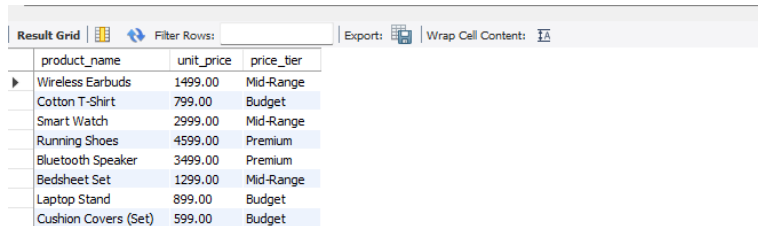
These questions test your understanding of conditional logic, database reliability principles, and transaction management.

Q24. Write a query using CASE to classify products into price tiers:

- 'Budget' : unit_price < 1000
- 'Mid-Range' : unit_price BETWEEN 1000 AND 3000
- 'Premium' : unit_price > 3000

Display: product_name, unit_price, price_tier.

```
SELECT product_name, unit_price, CASE WHEN unit_price < 1000 THEN 'Budget'
WHEN unit_price BETWEEN 1000 AND 3000 THEN 'Mid-Range' ELSE 'Premium'
END AS price_tier FROM products;
```

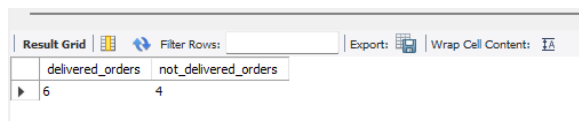


The screenshot shows a database interface with a 'Result Grid' tab selected. It displays the results of the SQL query from Q24. The grid has three columns: 'product_name', 'unit_price', and 'price_tier'. There are 10 rows of data, each representing a different product and its classification into a price tier.

product_name	unit_price	price_tier
Wireless Earbuds	1499.00	Mid-Range
Cotton T-Shirt	799.00	Budget
Smart Watch	2999.00	Mid-Range
Running Shoes	4599.00	Premium
Bluetooth Speaker	3499.00	Premium
Bedsheet Set	1299.00	Mid-Range
Laptop Stand	899.00	Budget
Cushion Covers (Set)	599.00	Budget

Q25. Using a CASE statement inside an aggregate function, count how many orders are 'Delivered' vs 'Not Delivered' (all other statuses). Display the result in a single row.

```
SELECT SUM(CASE WHEN status = 'Delivered' THEN 1 ELSE 0 END)
AS delivered_orders, SUM(CASE WHEN status <> 'Delivered' THEN 1 ELSE 0 END)
AS not_delivered_orders FROM orders;
```



The screenshot shows a database interface with a 'Result Grid' tab selected. It displays the results of the SQL query from Q25. The grid has two columns: 'delivered_orders' and 'not_delivered_orders'. There is one row of data showing the counts for each status.

delivered_orders	not_delivered_orders
6	4

Q26. Explain each letter of ACID:

- A – Atomicity
- C – Consistency
- I – Isolation
- D – Durability

Give a real-world example (e.g., bank transfer) showing why each property is important.

Answer:

ACID is a set of properties that ensures database transactions are processed correctly and reliably.

- **A – Atomicity:**
A transaction is completed fully or not at all. If any part of the transaction fails, all changes are rolled back.
- **C – Consistency:**
A transaction moves the database from one valid state to another, ensuring that all rules and constraints are maintained.
- **I – Isolation:**
Multiple transactions can run at the same time without affecting each other. One transaction should not interfere with another.
- **D – Durability:**
Once a transaction is committed, the changes are permanently saved, even if there is a system crash or power failure.

Real-World Example

Bank Transfer:

Suppose ₹5,000 is transferred from Account A to Account B.

- **Atomicity:** If the money is deducted from Account A but cannot be added to Account B due to an error, the entire transaction is cancelled, and the ₹5,000 is returned to Account A.
- **Consistency:** After the transfer, the total money in both accounts remains correct, and no invalid data is created.
- **Isolation:** If two people are transferring money at the same time, each transaction is processed independently without affecting the other.
- **Durability:** Once the transfer is completed successfully, it is permanently stored in the database and will not be lost even if the system crashes immediately afterward.

Q27. Write a SQL transaction that does the following atomically:

1. Insert a new order (order_id=1011, customer_id=102, today's date, 'Pending', 1598.00)
2. Insert two order items for that order
3. Update the stock_qty of the purchased products
4. If any step fails, ROLLBACK the entire transaction. Otherwise, COMMIT.

Write the complete BEGIN...COMMIT/ROLLBACK block.

```
START TRANSACTION;
    INSERT INTO orders VALUES (1054, 102, CURDATE(), 'Pending', 1598.00);
    INSERT INTO order_items VALUES (5043, 1011, 206, 1, 1299.00, 0);
    INSERT INTO order_items VALUES (5053, 1011, 208, 1, 299.00, 0);
    UPDATE products SET stock_qty = stock_qty - 1 WHERE product_id = 206;
    UPDATE products SET stock_qty = stock_qty - 1 WHERE product_id = 208;
COMMIT;
```

Output			
Action Output			
#	Time	Action	Message
1	17:10:33	START TRANSACTION	0 row(s) affected
2	17:10:33	INSERT INTO orders VALUES (1054, 102, CURDATE(), 'Pending', 1598.00)	1 row(s) affected
3	17:10:33	INSERT INTO order_items VALUES (5043, 1011, 206, 1, 1299.00, 0)	1 row(s) affected
4	17:10:33	INSERT INTO order_items VALUES (5053, 1011, 208, 1, 299.00, 0)	1 row(s) affected
5	17:10:33	UPDATE products SET stock_qty = stock_qty - 1 WHERE product_id = 206	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0
6	17:10:33	UPDATE products SET stock_qty = stock_qty - 1 WHERE product_id = 208	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0
7	17:10:33	COMMIT	0 row(s) affected